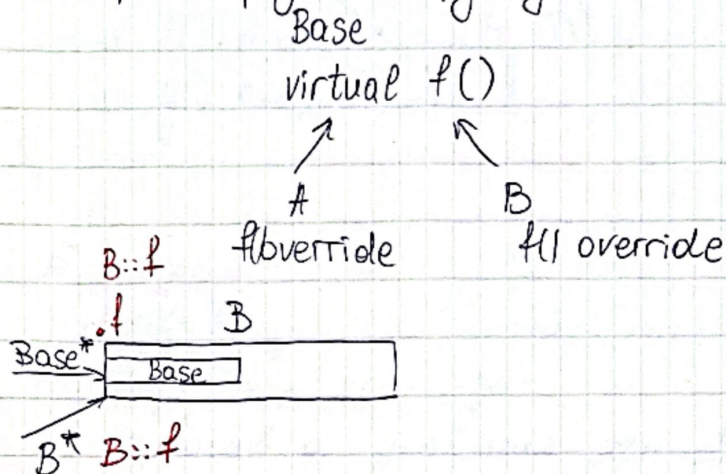


13.05.2024г.

лекция 12

Полиморфизъм

→ динамично свързване
чрез виртуални функции



Абстрактен клас

→ клас с поне 1 чисто виртуална функция

virtual void f() = 0;

⚠ Всеки наследник е длъжен да я имплементира

→ Възможно е да имат тело.

```
class A {
    virtual f() = 0;
    // има тело
}
```

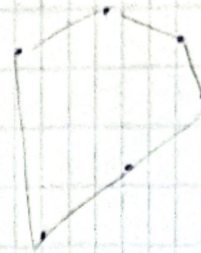
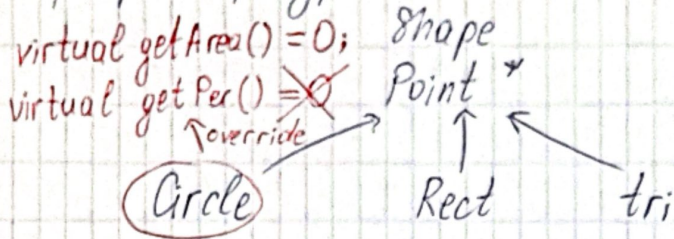
→ не можем да я извикаме извън класовете

```
class B: A {
    f() override {
    }
}
```

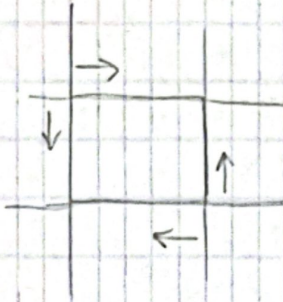
A::f() можем тук

```
{;
```

Пример с фигури:



virtual isPointIn = 0;



Примерна имплементация на динамично свързване

C++ стандарт:

Base * ptr = ...
ptr → f();

1. f(Base * this)
2. f(Der * this)
f(Der2 * this)

Ако f() е виртуална, ще се извика най-конкретната функция.

1. Да се намери правилната функция
2. Преобразуване указателя до правилния обект

Виртуални таблици

```
class A {
    virtual void f() { ... }
    virtual void g() { ... }
};
```

```
class B: A {
```

```
  void f() override { ... }
};
```

```
class C: B {
```

```
  void f() override { ... }
  void g() override { ... }
};
```

```
};
```

A

A B

A B C

```
A* ptr = ...;
```

```
ptr -> f(); // най-конкретното f()
```

Виртуална таблица

→ масив от указатели към функции

Всеки клас, който има виртуални функции, има своя виртуална таблица. В нея пише коя функция трябва да се извика.

vtable A;

A::f()
A::g()

A

vtable B;

B::f
A::g

A B

vtable C;

C::f
C::g

A B C

Всеки обект има виртуален указател, който сочи към виртуална таблица на класа

```
A* ptr = ...;
```

```
ptr -> f();
```

A

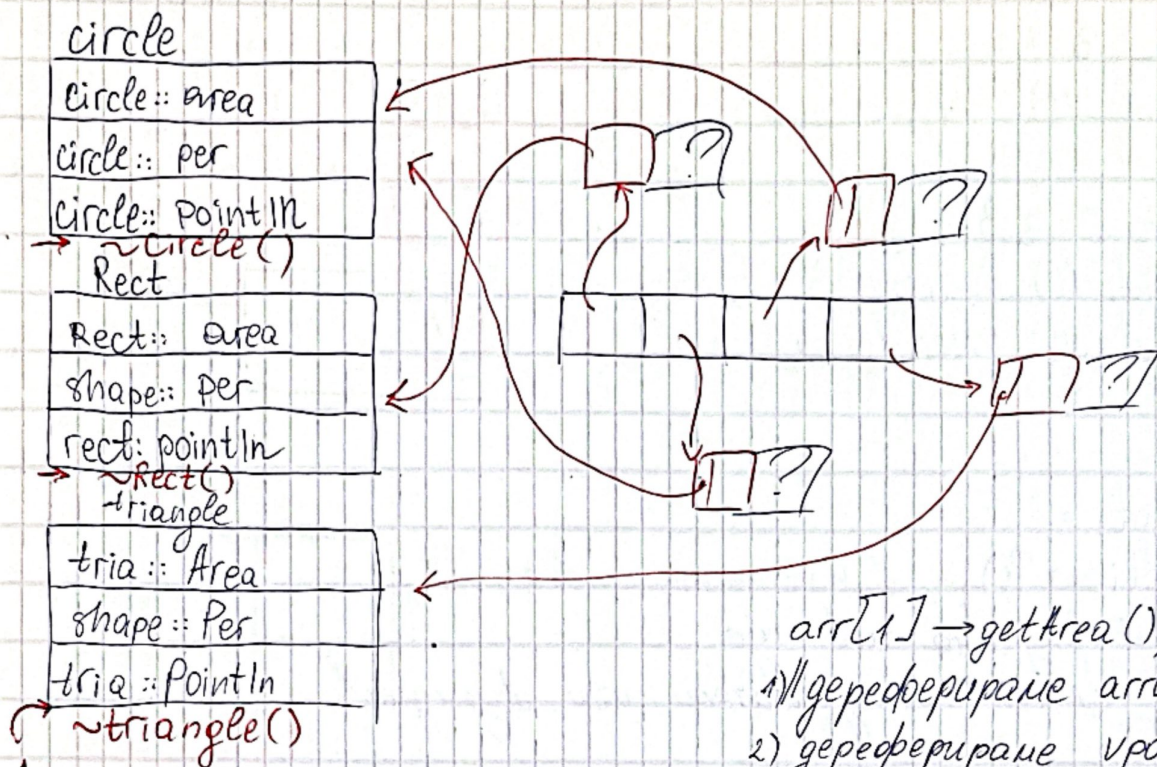
vpointer

C::f()

f(c* ptr)

подаваме ptr и той се
каства автоматично

// Виртуалните таблици
се създават compile-time

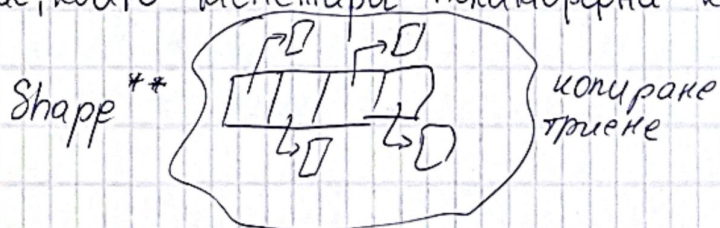


! имат и деструктори във виртуалните таблици, защото и деструкторите са вирт.
 // vpointer указателя заема място x32 → 4 байт.
 имат големина x64 → 8 байт.

arr[1] → getArea();
 1) // дерекференциране arr[1]
 2) дерекференциране vpointer
 3) извеждаме функцията с пререфиниращ указател

⚠ Не виртуалните функции не са във виртуалните таблици.

Клас, който менижира полиморфна колекция



```
class Container*
Shape** ptr;
Base
```

```
~Container() {
```

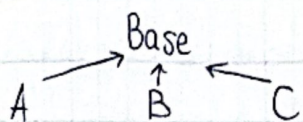
```
for i=0...n
```

```
delete ptr[i];
```

```
delete[] ptr;
```

```
}
```

→ деструктор Base е virtual



delete ptr[i];

virtual ~Base() → ~A { ... } ~Base() { ... }

// конструкторът и деструкторът на всеки клас
влият върху виртуалната таблица

A obj;

A(...) : Base(...)

Base да закъти
виртуалната таблица на A

~A() {

{ -> променя виртуалния ^{указател} деструктор на Base да не е A
~Base()

Base() {

...
{ } закътане vtable A

Хетерогенен

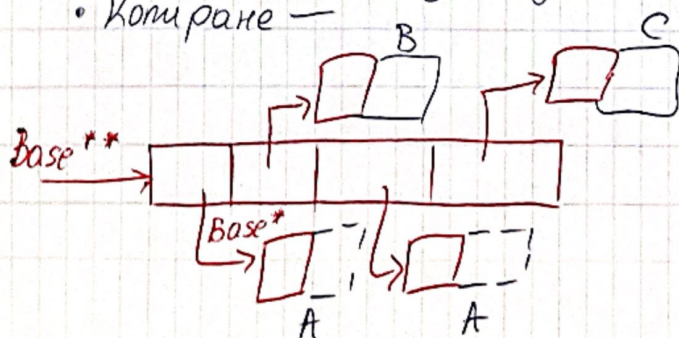
Контейнер

→ масив, от указатели към базов клас.

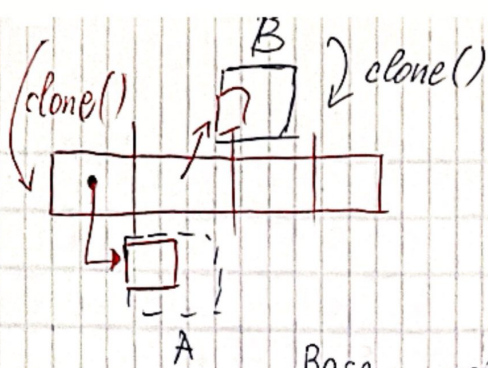
Пазите полиморфни обекти.

• Триене — вирт. деструктор

• Копиране —



клонирателна функция



virtual Base* clone() const;

A B

```
Base* A::clone override {
    return new A(*this);
}
```

```
Base* B::clone override {
    return new B(*this);
}
```

collection (const Collection & other)

// заделяне нов масив

```
for (int i=0; i<n; i++)
    data[i] = other.data[i] -> clone();
```

Пример за събеседване (изпит)

Клас Ферма

- Dog
- Cat
- Cow

Animal

virtual roar() = 0;

Фермата е хетер. контейнер.

- roarAll();

- добавяне на животно - две функции за това
- едната е bool

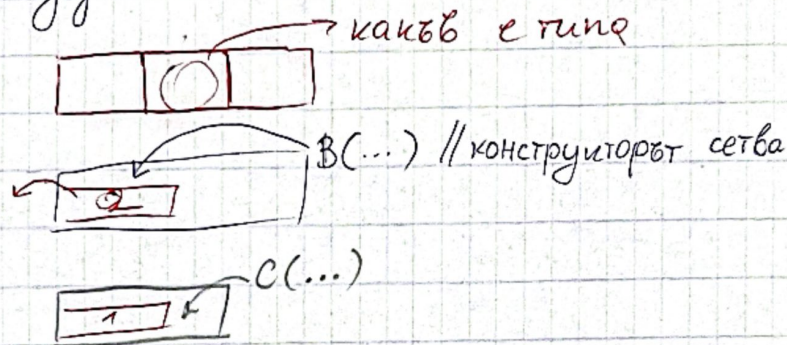
- getSize

- голяма шестича

- глен, функция, приема индекс и връща типа на обекта

Иначе: `dynamic_cast`

Иначе: В базовия клас да запазваме, какъв е типа на текущия обект



Параметричен полиморфизъм

→ Функция / Клас с общо предназначение спрямо типовете.

Шаблони / Templates

```
int max(int a, int b) {
    return a > b ? a : b;
}
```

```
const string& max(const string& a,
                  const string& b) {
    return a > b ? a : b;
}
```

// двете функции правят едно и също

Целта е да напишем функцията 1 път

```
template <typename T>
const T& max(const T& lhs,
             const T& rhs) {
    return lhs > rhs ? lhs : rhs;
}
```

//извикване

max<int>(3, 4)

→ създава копие на
шаблонния код и
замества T с конкретен
тип

max<char>(...)

⚠ Compile-time полиморфизъм
- типовете трябва да са известни по време на
за шаблоните компилация